

ROS2 Service & Action

서비스·액션 서버/클라이언트 구현

cchyun@gmail.com

Part 1. 서비스(Service)

서버/클라이언트 구현

서비스(Service)란?

- 서비스는 1:1 통신 방식으로, 요청(Request)을 보내고 반드시 응답(Response)을 받는 동기적/비동기적 RPC 패턴
 - 지속적인 데이터 흐름이 필요한 토픽(Topic) 과 달리, 서비스는 특정 상황에 적합
 - 토픽 vs 서비스
 - 토픽: 지속적 데이터 스트림 (예: 센서 데이터)
 - 서비스: 요청-응답 구조 (예: LED 켜기, IK 계산 요청)

서비스가 적합한 상황 3가지

- **로봇 상태 설정 및 쿼리**
 - 노드의 현재 설정을 확인하거나 특정 모드(절전, 대기)로 즉시 변경해야 할 때
- **일회성 복잡한 계산**
 - 역기구학(IK) 계산이나 경로 계획 요청처럼, 결과값이 나올 때까지 기다려야 하는 짧은 작업
- **장치 제어 명령 (LED, 그리퍼 등)**
 - LED 켜기, 그리퍼 닫기 명령 후 해당 동작이 실제로 **성공했는지 즉시 확인**이 필요한 경우

- 서비스 인터페이스는 `.srv` 파일에 정의하며, `---` 구분자로 요청/응답을 분리

```
# LedControl.srv

# Request 필드: 클라이언트가 보낼 데이터
bool state # true: 켜기, false: 끄기
---
# Response 필드: 서버가 답할 데이터
bool success # 제어 성공 여부
string message # 결과 메시지
```

- Python 자동 생성 클래스 (colcon 빌드 후)
 - `LedControl.Request` : 요청 데이터를 담는 객체
 - `LedControl.Response` : 응답 데이터를 담는 객체
 - `LedControl` : 위 두 클래스를 포함하는 상위 서비스 인터페이스 클래스

(실습) 서비스 인터페이스 패키지 생성

- 1. 워크스페이스 이동

```
cd ~/Workspace/ros2_ws/src
```

- 2. 인터페이스 패키지 생성 (ament_cmake)

```
ros2 pkg create --build-type ament_cmake my_robot_interfaces
```

- 3. srv 파일 생성

```
mkdir -p my_robot_interfaces/srv  
# LedControl.srv 파일 작성 (--- 로 요청/응답 분리)
```

- 빌드 시스템에 srv 파일 등록

```
cmake_minimum_required(VERSION 3.8)
project(my_robot_interfaces)

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "srv/LedControl.srv"
)

ament_package()
```

- package.xml 의존성 추가

```
<buildtool_depend>ament_cmake</buildtool_depend>
<depend>rosidl_default_generators</depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

- Python 패키지 생성 (의존성 포함)

```
cd ~/Workspace/ros2_ws/src
ros2 pkg create --build-type ament_python \
  my_robot_service \
  --dependencies rclpy my_robot_interfaces
```

- 디렉토리 구조

```
my_robot_service/
├── my_robot_service/
│   ├── __init__.py
│   ├── led_server.py      # 서버 노드
│   └── led_client.py      # 클라이언트 노드
├── setup.py
└── package.xml
```


- 서버는 클라이언트의 요청을 기다렸다가 지정된 로직을 수행하고 응답을 돌려줌

- `create_service()` 파라미터

- `srv_type` : 사용할 서비스 인터페이스 클래스 (예: `LedControl`)
- `srv_name` : 서비스의 이름 (예: `'set_led'`)
- `callback` : 요청이 왔을 때 실행할 함수

- 콜백 처리 패턴

- 서버 콜백 함수는 항상 `(request, response)` 두 개의 인자를 받음
- `request`에서 데이터를 읽어 로직을 수행한 후, `response` 객체 필드 값을 수정하여 반환

서비스 서버 전체 코드 (LED 제어)

```
import rclpy
from rclpy.node import Node
from my_robot_interfaces.srv import LedControl

class LedServiceServer(Node):
    def __init__(self):
        super().__init__('led_service_server')
        self.srv = self.create_service(
            LedControl, 'set_led', self.set_led_callback)
        self.get_logger().info('LED 서비스 서버가 시작되었습니다.')

    def set_led_callback(self, request, response):
        if request.state:
            self.get_logger().info('LED 켜기 요청 수신')
            response.success = True
            response.message = "LED를 성공적으로 켜했습니다."
        else:
            self.get_logger().info('LED 끄기 요청 수신')
            response.success = True
            response.message = "LED를 성공적으로 껐습니다."
        return response  # 반드시 response 반환!

def main():
    rclpy.init()
    node = LedServiceServer()
    rclpy.spin(node)
    rclpy.shutdown()
```

- 클라이언트는 서버에 요청을 보내고 응답이 올 때까지 기다림
 - `create_client(srv_type, srv_name)`
 - 서버와 이름 및 타입이 일치해야 통신 가능
 - `wait_for_service(timeout_sec=1.0)`
 - 서버가 네트워크 상에 존재하지 않을 때 요청을 보내면 에러 발생
 - 서버가 준비될 때까지 대기하는 코드가 필수
 - `call_async()` + `spin_until_future_complete()`
 - `call_async()` : 요청을 보낸 후 즉시 **Future** 객체를 반환 (비동기 방식)
 - 메인 루프를 차단하지 않기 위해 권장되는 방식
 - `spin_until_future_complete()` : 응답이 올 때까지 대기

서비스 클라이언트 전체 코드 (LED 제어)

```
import rclpy
from rclpy.node import Node
from my_robot_interfaces.srv import LedControl

class LedServiceClient(Node):
    def __init__(self):
        super().__init__('led_service_client')
        self.cli = self.create_client(LedControl, 'set_led')
        # 서버가 준비될 때까지 1초마다 확인하며 대기
        while not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('서버 대기 중...')
        self.req = LedControl.Request()

    def send_request(self, state):
        self.req.state = state
        self.future = self.cli.call_async(self.req)      # 비동기 요청 - 결과를 기다리지 않고 즉시 반환
        rclpy.spin_until_future_complete(self, self.future) # future 완료까지 블로킹 대기
        return self.future.result()

def main():
    rclpy.init()
    client = LedServiceClient()
    response = client.send_request(True)
    client.get_logger().info(
        f'결과: {response.success}, 메시지: {response.message}')
    client.destroy_node()
    rclpy.shutdown()
```

- 콘솔 실행 명령 등록

```
from setuptools import setup

setup(
    name='my_robot_service',
    version='0.0.0',
    packages=['my_robot_service'],
    # ... data_files, install_requires ...
    entry_points={
        'console_scripts': [
            'led_server = my_robot_service.led_server:main',
            'led_client = my_robot_service.led_client:main',
        ],
    },
)
```

- 빌드 및 환경 설정

```
cd ~/Workspace/ros2_ws
colcon build
source install/setup.bash
```

- 터미널 1 — 서버 실행

```
source ~/Workspace/ros2_ws/install/setup.bash  
ros2 run my_robot_service led_server
```

- 터미널 2 — 클라이언트 실행

```
source ~/Workspace/ros2_ws/install/setup.bash  
ros2 run my_robot_service led_client
```

- 터미널에서 서비스 상태를 확인하고 직접 호출 가능
 - 서비스 목록 확인 : 현재 활성화된 모든 서비스를 리스팅

```
ros2 service list
```

- 서비스 직접 호출 : 코드 작성 없이 서버를 테스트할 때 매우 유용

```
ros2 service call /set_led my_robot_interfaces/srv/LedControl "{state: true}"
```

- 서버가 응답을 주지 않을 경우 클라이언트가 무한정 대기하는 것을 방지해야 함

- 1. 연결 대기 타임아웃

- `wait_for_service(timeout_sec=5.0)` 으로 대기 시간 제한
- 서버 부재 시 예외 처리 로직 구현 가능

- 2. 호출 타임아웃

- `future` 객체를 주기적으로 확인하거나, `timer` 를 활용
- 일정 시간 내에 `future.done()` 이 `True` 가 되지 않으면 호출 취소

입문자가 자주 하는 실수 (서비스)

- 서비스 이름/타입 불일치

- 클라이언트와 서버의 이름이나 `.srv` 타입이 하나라도 다르면 서로를 찾지 못함

- 콜백 내 동기 호출 (Deadlock)

- 서버 콜백 함수 안에서 다른 서비스를 호출하고 응답을 기다리면 **데드락** 발생

- 서버 미응답 대기

- 서버가 켜져 있지 않은 상태에서 요청을 보내면 클라이언트가 멈춰 있는 것처럼 보임
- 항상 `wait_for_service` 를 먼저 실행할 것

- Response 미반환

- 서버 콜백 함수 마지막에 `return response` 를 누락하면 클라이언트는 응답을 영원히 받지 못함

Part 2. 액션(Action)

서버/클라이언트 구현

액션(Action)이란?

- ROS2 액션은 서비스와 유사한 요청-응답 구조를 가지면서도, 장기 실행 작업을 처리하기 위해 피드백과 취소 기능을 제공하는 복합 통신 프로토콜
 - 서비스(Service) : 즉각적으로 끝나는 작업에 적합 (클라이언트는 응답이 올 때까지 대기)
 - 액션(Action) : 완료까지 수 초~수 분이 걸리는 **장기 실행 작업**에 필수
- 액션의 3가지 핵심 기능
 - 목표(Goal) : 클라이언트가 서버에 요청하는 작업
 - 피드백(Feedback) : 작업 수행 중 주기적으로 전달하는 중간 정보
 - 결과(Result) : 작업이 완전히 끝난 후 전달하는 최종 결과

액션이 서비스보다 적합한 상황 (Nav2 시나리오)

- 장기 실행 작업

- 로봇의 이동처럼 완료까지 수 초에서 수 분이 걸리는 작업
- 비동기 처리가 필요하며, 도중의 진행 상황을 알 필요가 있음

- 진행률 피드백

- 자율주행 중 목적지까지 남은 거리나 예상 도착 시간을 주기적으로 확인

- 목표 취소 및 수정

- 이동 중 장애물 발견 시 기존 작업을 취소(Cancel) 하거나 새로운 목표로 전환(Preempt)

- Nav2에서 액션이 필수인 이유

- 로봇이 목적지로 가는 과정이 길고, 지속적인 피드백으로 경로 수정이 필요
- 필요 시 즉각 이동을 멈출 수 있어야 함

- `.action` 파일은 세 부분으로 나뉘며, 각 부분은 `---`로 구분
 - **Goal** : 클라이언트가 서버에 요청하는 목표 데이터
 - **Result** : 작업이 완전히 끝난 후 서버가 클라이언트에 전달하는 최종 결과
 - **Feedback** : 작업 수행 중에 주기적으로 클라이언트에 보내는 중간 정보

```
# MoveRobot.action

float32 target_distance # Goal: 이동할 목표 거리
---
bool reached           # Result: 목표 도달 성공 여부
---
float32 current_distance # Feedback: 현재까지 이동한 거리
```

(실습) 액션 인터페이스 추가

- 1. action 파일 생성

```
mkdir -p ~/Workspace/ros2_ws/src/my_robot_interfaces/action  
# MoveRobot.action 파일 작성
```

- 2. CMakeLists.txt 수정 (action 추가)

```
rosidl_generate_interfaces(${PROJECT_NAME}  
  "srv/LedControl.srv"  
  "action/MoveRobot.action"    # ← action 파일 추가  
)
```

- 3. 패키지 재빌드

```
cd ~/Workspace/ros2_ws  
colcon build --packages-select my_robot_interfaces  
source install/setup.bash
```

(실습) 액션 서버/클라이언트 패키지 생성

- Python 패키지 생성 (rclpy_action 의존성 포함)

```
cd ~/Workspace/ros2_ws/src
ros2 pkg create --build-type ament_python \
  my_robot_action \
  --dependencies rclpy rclpy_action my_robot_interfaces
```

- 디렉토리 구조

```
my_robot_action/
├── my_robot_action/
│   ├── __init__.py
│   ├── move_server.py      # 액션 서버 노드
│   └── move_client.py      # 액션 클라이언트 노드
├── setup.py
└── package.xml
```

- 액션 서버는 `rclpy.action.ActionServer` 객체를 통해 생성
 - 목표 수락/거절
 - 서버는 목표 요청이 들어오면 이를 수락하거나 거절 가능
 - `goal_callback`을 구현하여 조건을 검사할 수 있음 (기본: 항상 수락)
 - 피드백 주기적 발신
 - `execute_callback` 내부 루프에서 `goal_handle.publish_feedback()` 호출
 - 상태 처리
 - 작업 완료 → `goal_handle.succeed()` 호출
 - 취소 요청 → `goal_handle.canceled()` 처리
 - 작업 수행 불가 → `goal_handle.abort()` 처리

액션 서버 전체 코드

```
import time
import rclpy
from rclpy.action import ActionServer
from rclpy.node import Node
from my_robot_interfaces.action import MoveRobot

class RobotMoveServer(Node):
    def __init__(self):
        super().__init__('robot_move_server')
        self._action_server = ActionServer(
            self, MoveRobot, 'move_robot', self.execute_callback)

    def execute_callback(self, goal_handle):
        self.get_logger().info('목표 거리 이동 시작...')
        feedback_msg = MoveRobot.Feedback()
        feedback_msg.current_distance = 0.0
        target = goal_handle.request.target_distance

        for i in range(1, int(target) + 1):
            if goal_handle.is_cancel_requested: # 취소 확인
                goal_handle.canceled()
                return MoveRobot.Result(reached=False)
            time.sleep(1.0)
            feedback_msg.current_distance = float(i)
            goal_handle.publish_feedback(feedback_msg)
            self.get_logger().info(f'진행 중: {i}/{target}')

        goal_handle.succeed()
        result = MoveRobot.Result()
        result.reached = True
        return result

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(RobotMoveServer())
    rclpy.shutdown()
```

- 액션 클라이언트는 서버에 목표를 전달하고 결과와 피드백을 수신
 - `send_goal_async()`
 - 서버가 목표를 수행하는 동안 클라이언트 노드가 멈추지 않도록 **비동기 방식**으로 목표 전송
 - **피드백 콜백 등록**
 - `send_goal_async()`의 인자로 `feedback_callback`을 전달
 - 피드백 메시지가 올 때마다 특정 함수가 실행되도록 등록
 - **목표 취소**
 - `goal_handle.cancel_goal_async()`를 호출하여 실행 중인 작업을 중단

액션 클라이언트 전체 코드

```
import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node
from my_robot_interfaces.action import MoveRobot

class RobotMoveClient(Node):
    def __init__(self):
        super().__init__('robot_move_client')
        self._action_client = ActionClient(self, MoveRobot, 'move_robot')

    def send_goal(self, distance):
        goal_msg = MoveRobot.Goal()
        goal_msg.target_distance = distance
        self._action_client.wait_for_server()
        self._send_goal_future = self._action_client.send_goal_async(
            goal_msg, feedback_callback=self.feedback_callback)
        self._send_goal_future.add_done_callback(self.goal_response_callback)

    def goal_response_callback(self, future):
        goal_handle = future.result()
        if not goal_handle.accepted:
            self.get_logger().info('서버가 목표를 거절했습니다.')
            return
        self.get_logger().info('목표가 수락되었습니다.')
        self._get_result_future = goal_handle.get_result_async()
        self._get_result_future.add_done_callback(self.get_result_callback)

    def feedback_callback(self, feedback_msg):
        distance = feedback_msg.feedback.current_distance
        self.get_logger().info(f'피드백: 현재 이동 거리 {distance}')

    def get_result_callback(self, future):
        result = future.result().result
        self.get_logger().info(f'최종 결과: 도달 완료 = {result.reached}')
        rclpy.shutdown()

def main(args=None):
    rclpy.init(args=args)
    node = RobotMoveClient()
    node.send_goal(5.0)
    rclpy.spin(node)
```

- 콘솔 실행 명령 등록

```
from setuptools import setup

setup(
    name='my_robot_action',
    version='0.0.0',
    packages=['my_robot_action'],
    # ... data_files, install_requires ...
    entry_points={
        'console_scripts': [
            'move_server = my_robot_action.move_server:main',
            'move_client = my_robot_action.move_client:main',
        ],
    },
)
```

- 빌드 및 환경 설정

```
cd ~/Workspace/ros2_ws
colcon build --packages-select my_robot_interfaces my_robot_action
source install/setup.bash
```

- 터미널 1 — 액션 서버 실행

```
source ~/Workspace/ros2_ws/install/setup.bash  
ros2 run my_robot_action move_server
```

- 터미널 2 — 액션 클라이언트 실행

```
source ~/Workspace/ros2_ws/install/setup.bash  
ros2 run my_robot_action move_client
```

- 터미널에서 액션 상태를 확인하고 직접 목표를 전송 가능

- 액션 목록 확인 : 현재 활성화된 액션 목록 확인

```
ros2 action list
```

- 액션 직접 호출 : 피드백을 보려면 `--feedback` 옵션 추가
- 형식: `ros2 action send_goal <액션이름> <액션타입> "{데이터}"`

```
ros2 action send_goal /move_robot my_robot_interfaces/action/MoveRobot \  
  "{target_distance: 5.0}" --feedback
```

입문자가 자주 하는 실수 (액션)

- 목표 상태 미설정

- `execute_callback`에서 명시적으로 `succeed()`를 호출하지 않으면
- ROS2는 해당 작업을 자동으로 **ABORTED(중단)** 상태로 처리

- 의존성 누락

- `package.xml`에 `rclpy_action` 및 사용자 정의 인터페이스 패키지 의존성 미추가
- 빌드 에러가 발생하는 경우가 많음

- 단일 스레드 차단

- 루프 안에서 `time.sleep()` 등을 사용하여 노드 전체를 멈추면
- 피드백 전송이나 취소 요청 처리가 지연될 수 있음

Part 3. 실습 과제

서비스 & 액션 직접 구현하기

- **요구사항:** 두 정수를 입력받아 합을 반환하는 서비스를 구현하세요.

- **Step 1. 인터페이스 정의** (`AddTwoInts.srv`)

```
int64 a
int64 b
---
int64 sum
```

- **Step 2. 서버 코드 작성**

```
import rclpy
from rclpy.node import Node
from my_robot_interfaces.srv import AddTwoInts

class AddServer(Node):
    ...
```

- Step 3. 클라이언트 코드 작성

```
import rclpy
from rclpy.node import Node
from my_robot_interfaces.srv import AddTwoInts

class AddClient(Node):
    ...
```

실습 1 — 덧셈 서비스 (실행)

- Step 4. 빌드 및 실행

터미널 1 — 서버 실행

```
ros2 run my_robot_service add_server
```

터미널 2 — 클라이언트 실행

```
ros2 run my_robot_service add_client
```

터미널 3 — CLI 테스트

```
ros2 service call /add_two_ints my_robot_interfaces/srv/AddTwoInts "{a: 7, b: 3}"
```

- 기대 출력

```
[add_server] 계산: 5 + 10 = 15
```

```
[add_client] 결과: 15
```

실습 2 — 카운트다운 액션 구현

- **요구사항:** 목표 숫자까지 1초 간격으로 카운트하며 피드백을 전송하는 액션을 구현하세요.
- **Step 1. 인터페이스 정의 (`Countdown.action`)**

```
int32 target_number      # Goal: 카운트할 목표 숫자
---
bool success            # Result: 완료 여부
---
int32 current_number     # Feedback: 현재 숫자
```

- Step 2. 서버 코드 작성

```
import time
import rclpy
from rclpy.action import ActionServer
from rclpy.node import Node
from my_robot_interfaces.action import Countdown

class CountdownServer(Node):
    ...
```

- Step 3. 클라이언트 코드 작성

```
import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node
from my_robot_interfaces.action import Countdown

class CountdownClient(Node):
    ...
```

실습 2 — 카운트다운 액션 (실행)

- Step 4. 빌드 및 실행

```
# 터미널 1 - 서버 실행
ros2 run my_robot_action countdown_server

# 터미널 2 - 클라이언트 실행
ros2 run my_robot_action countdown_client

# 터미널 3 - CLI 테스트 (피드백 포함)
ros2 action send_goal /countdown my_robot_interfaces/action/Countdown \
    "{target_number: 5}" --feedback
```

- 기대 출력

```
[countdown_server] 카운트: 1/5
[countdown_server] 카운트: 2/5
[countdown_client] 피드백: 현재 1
[countdown_client] 피드백: 현재 2
...
[countdown_client] 완료: success=True
```



기본 실습을 완료했다면 다음을 시도해보세요.

도전 1 — 서비스 타임아웃 처리

- 클라이언트에서 `wait_for_service(timeout_sec=3.0)` 를 사용
- 서버가 없을 때 적절한 에러 메시지 출력

도전 2 — 액션 Goal 거절 조건

- `goal_callback` 을 추가하여 `target_number > 10` 일 때 거절

도전 3 — 액션 취소 테스트

- 클라이언트 실행 중 `Ctrl+C` 로 취소
- 서버가 `is_cancel_requested` 를 감지하는지 확인